



Universidade do Minho
Escola de Engenharia

Departamento de Informática

Licenciatura em Engenharia Informática

Processamento de Linguagens

Trabalho Prático

Grupo n.º 28

Carlos Daniel Lopes Cunha, n.º A106910

Francisco Queirós Ribeiro da Costa Maia, n.º A108962

Índice

1	Introdução	1
2	Opções de Implementação e Arquitetura	2
2.1	Pré-processamento	2
2.2	Análise Léxica e Sintática	2
2.2.1	<i>Análise Léxica</i>	2
2.2.2	<i>Análise Sintática e Construção da AST</i>	3
2.2.3	<i>Reestruturação Pós-Parse da AST</i>	3
2.3	Análise Semântica	3
2.3.1	<i>Gestão da Tabela de Símbolos</i>	3
2.3.2	<i>Tipagem Implícita</i>	4
2.3.3	<i>Verificações de Coerência e Tipos</i>	4
2.4	Geração de Código (Máquina Virtual)	4
2.4.1	<i>Alocação de Memória</i>	4
2.4.2	<i>Controlo de Fluxo</i>	5
2.4.3	<i>Resolução de Operações Complexas</i>	5
3	Gramática Utilizada	5
3.1	Especificação BNF	5
3.2	Resolução de Ambiguidades e Precedência	6
4	Testes Implementados	7
4.1	Casos de Teste	7
4.2	Automação da Bateria de Testes	7
5	Instruções de Utilização	8
5.1	Requisitos de Instalação	8
5.2	Compilação de Ficheiros	8
5.3	Execução na Máquina Virtual	8
5.4	Execução dos Testes	8
6	Dificuldades Encontradas	9
7	Conclusão	10

1 Introdução

O presente relatório descreve o desenvolvimento de um compilador para a linguagem *Fortran 77* (standard ANSI X3.9-1978), realizado no âmbito da unidade curricular de Processamento de Linguagens. O objetivo deste projeto consistiu na construção de uma ferramenta capaz de traduzir código fonte Fortran para um conjunto de instruções compatíveis com uma Máquina Virtual (VM) fornecida pelos professores.

A arquitetura do compilador foi concebida seguindo as fases clássicas de compilação, estruturadas num *pipeline* modular em Python. O processo inicia-se com um pré-processador, encarregue de normalizar o código fonte, seguido das fases de análise léxica e sintática recorrendo à biblioteca *PLY* (*Python Lex-Yacc*). Após o *parsing*, uma fase de reestruturação pós-parse organiza a AST produzida, aninhando os corpos dos ciclos `DO` com base nos seus *labels*. Segue-se a análise semântica, que garante a coerência de tipos e escopos, culminando na geração de código intermédio para a VM.

2 Opções de Implementação e Arquitetura

A arquitetura do compilador foi desenhada para ser modular, facilitando a correção de erros e a manutenção de cada fase do pipeline. Nesta secção, detalham-se as principais escolhas técnicas e a lógica por trás dos componentes desenvolvidos.

2.1 Pré-processamento

Uma das dificuldades sintáticas do *Fortran 77* está na coexistência de dois formatos de escrita: o formato de colunas fixas (*fixed-form*) e o formato livre (*free-form*). Para resolver este problema, optamos pela implementação de um pré-processador dedicado (`src/preprocessor.py`), que atua antes de o código chegar ao analisador léxico.

O pré-processador realiza as seguintes transformações:

- **Normalização de Comentários:** Identifica e remove linhas de comentário que começam por `C`, `*` ou `!` (suporta tanto o padrão clássico como extensões comuns), evitando que assim estas interfiram na construção da AST.
- **Junção de Linhas de Continuação:** O *Fortran 77* utiliza a sexta coluna para indicar a continuação de uma instrução. O pré-processador deteta estes caracteres e junta as linhas fragmentadas numa única linha de código.
- **Gestão de Espaços e *Case-Insensitivity*:** Como a linguagem é insensível a espaços em branco e a *case* (maiúsculas/minúsculas), o pré-processador normaliza o texto para maiúsculas e remove espaços redundantes em locais onde estes não têm significado semântico.
- **Preservação de Labels:** Os *labels* numéricos, essenciais para comandos como `GOTO` e ciclos `DO`, são extraídos e isolados para que o *lexer* os possa identificar como *tokens* distintos de forma imediata.

Esta abordagem permitiu que as fases seguintes (léxica e sintática) fossem mais simples, uma vez que estas trabalham sobre um fluxo de texto já limpo e estruturado, independentemente do formato original utilizado pelo programador.

2.2 Análise Léxica e Sintática

A tradução do fluxo de caracteres para uma estrutura lógica assenta na biblioteca *PLY*:

2.2.1 Análise Léxica

O analisador léxico (`src/lexer.py`) define o conjunto de *tokens* necessários para representar a linguagem. Foram implementadas expressões regulares para identificar:

- **Palavras-chave:** Identificação de comandos estruturais como `PROGRAM`, `SUBROUTINE`, `INTEGER`, `DO`, `IF`, `THEN`, `ELSE`, `ENDIF`, entre outros. As funções intrínsecas mais comuns (`MOD`, `ABS`, `MAX`, `MIN`, `SQRT`, etc.) são também tratadas como palavras-chave para simplificar o reconhecimento sintático.

- **Operadores e Delimitadores:** Incluindo operadores aritméticos (+, -, *, /, **), relacionais (.EQ., .NE., .LT., etc.) e lógicos (.AND., .OR., .NOT.).
- **Literais e Identificadores:** *Regex* específicas para números inteiros, reais (ponto flutuante) e cadeias de caracteres, além da regra para identificadores de variáveis e subprogramas.

Uma particularidade implementada foi o tratamento de *labels* numéricos no início das instruções, que são capturados como *tokens* específicos para suportar a lógica de saltos e ciclos na fase sintática.

2.2.2 Análise Sintática e Construção da AST

O *parser* (`src/parser.py`) utiliza uma gramática LALR (Look-Ahead LR) para validar a estrutura do programa. À medida que as regras gramaticais são reconhecidas, o compilador não se limita a validar a sintaxe, mas constrói também uma **Árvore Sintática Abstrata (AST)**.

Cada nó da árvore é instanciado a partir de classes definidas em `src/ast_nodes.py`, representando construções como atribuições (**Assignment**), ciclos (**DoLoop**), ou chamadas de subprogramas (**CallStatement**). Esta representação intermédia é fundamental, pois isola a análise semântica e a geração de código das "manias" da sintaxe concreta do Fortran.

Para resolver ambiguidades clássicas, como o problema do *dangling else* em blocos IF, foram definidas regras de precedência e associatividade no *Yacc*, garantindo que a árvore gerada reflète a precedência de operadores e a hierarquia de blocos da linguagem original.

2.2.3 Reestruturação Pós-Parse da AST

O *parser* LALR produz inicialmente uma lista **plana** de instruções. No entanto, os ciclos DO do Fortran 77 não têm delimitadores explícitos no fim, o seu corpo é delimitado por um *label* numérico num CONTINUE. Para reconstruir a hierarquia correta da AST, foi implementado o módulo `src/structure.py`.

Este módulo percorre a lista de instruções e, para cada DoLoop encontrado, recolhe as instruções subsequentes até encontrar o **ContinueStatement** com o *label* correspondente, atribuindo-as ao campo *body* do ciclo. O processo é recursivo, suportando ciclos DO aninhados. Caso o CONTINUE do fim esteja em falta, é lançado um erro com indicação do *label* em falta, o que constitui a principal verificação de coerência estrutural do compilador.

2.3 Análise Semântica

A fase de análise semântica (`src/semantic.py`) é responsável por validar o significado do programa e garantir coerência interna antes da geração de código. Esta etapa percorre a AST e interage com uma estrutura de dados central: a Tabela de Símbolos.

2.3.1 Gestão da Tabela de Símbolos

O compilador implementa uma gestão de contextos hierárquica. Existe uma tabela de símbolos global para registar o nome do programa principal e dos subprogramas

(SUBROUTINE e FUNCTION), e tabelas de símbolos locais para cada um desses blocos. Cada entrada na tabela armazena metadados críticos:

- **Tipo de Dado:** Inteiro, Real ou Lógico.
- **Categoria:** Variável simples, *array* (com as respectivas dimensões) ou parâmetro de função.
- **Endereçamento:** O desvio (*offset*) necessário para a alocação de memória na Máquina Virtual.

2.3.2 Tipagem Implícita

Uma das particularidades do *Fortran 77* é a regra de tipagem implícita, que o analisador semântico respeita rigorosamente. Caso uma variável não seja declarada explicitamente numa instrução `INTEGER` ou `REAL`, o compilador atribui o tipo com base na letra inicial do identificador:

- Identificadores iniciados por I, J, K, L, M, N são classificados como `INTEGER`.
- Todos os restantes são classificados como `REAL`.

2.3.3 Verificações de Coerência e Tipos

Durante a travessia da árvore, o analisador realiza as seguintes validações:

- **Compatibilidade de Tipos:** Infere os tipos das expressões e garante a promoção automática de `INTEGER` para `REAL` em operações mistas, sinalizando conflitos semânticos quando detetados.
- **Redeclaração de Variáveis:** Quando uma variável já registada na tabela de símbolos é declarada novamente com um tipo explícito diferente (e.g., um parâmetro implicitamente tipado depois refinado por uma declaração `INTEGER`), o tipo é atualizado na tabela, permitindo o padrão comum do Fortran de declarar parâmetros e depois refinar os seus tipos.
- **Verificação Estrutural dos Ciclos DO:** A correspondência entre o *label* de um ciclo `DO` e o seu `CONTINUE` é validada durante a reestruturação pós-parse (`src/structure.py`), com emissão de erro semântico caso o `CONTINUE` do final esteja ausente.

2.4 Geração de Código (Máquina Virtual)

A última fase do compilador (`src/codegen.py`) consiste na tradução da AST em instruções da Máquina Virtual fornecida. Este processo é realizado através de uma travessia em profundidade (*depth-first*) da árvore, emitindo sequencialmente os *opcodes* correspondentes a cada nó.

2.4.1 Alocação de Memória

A geração de código interage com a Tabela de Símbolos calculada na fase semântica. O endereçamento das variáveis é gerido da seguinte forma:

- As variáveis globais são instanciadas e acedidas através de instruções `PUSHG` e `STOREG`, usando *offsets* estáticos calculados na tabela de símbolos.

- O compilador implementa o conceito de *frames* de execução. No início de cada subprograma, é alocado espaço para as variáveis locais através da instrução `PUSHN`. As variáveis e parâmetros locais são acedidos com `PUSHL/STOREL` por *offset*. No final da execução, a instrução `RETURN` devolve o controlo a quem chamou, preservando o isolamento de escopos.

2.4.2 Controlo de Fluxo

As estruturas de controlo do Fortran, como `IF-THEN-ELSE` e os ciclos `DO`, são mapeadas para instruções de salto condicional (`JZ` - *Jump if Zero*) e incondicional (`JUMP`). A associação entre os *labels* numéricos do Fortran e as etiquetas geradas para a VM faz com que o fluxo do programa original seja respeitado, tirando partido da validação estrutural prévia que assegura o fecho correto dos blocos, como por exemplo, o emparelhamento entre `DO` e `CONTINUE`.

2.4.3 Resolução de Operações Complexas

Devido à ausência de instruções nativas na VM para certas operações matemáticas comuns no Fortran, foi adotada uma estratégia de alocação de endereços de memória temporários numa zona reservada acima das variáveis do programa principal:

- **Operação de Potência (**):** Como a VM não suporta exponenciação direta, o compilador gera um bloco de código *inline* que utiliza variáveis temporárias para implementar um ciclo de multiplicações sucessivas correspondentes ao expoente.
- **Funções Intrínsecas (MAX e MIN):** A avaliação do maior ou menor valor entre expressões foi implementada utilizando as instruções de comparação da VM (`SUP` e `INF`). O resultado destas comparações é utilizado para ditar saltos condicionais que armazenam o valor correto num registo temporário, que é posteriormente colocado no topo da pilha para continuar a avaliação da expressão.

3 Gramática Utilizada

O analisador sintático foi implementado com recurso ao gerador de *parsers* LALR(1) disponibilizado pela biblioteca PLY (`ply.yacc`). A "gramática livre de contexto" desenhada para este compilador abrange as construções essenciais do *Fortran 77*, bem como as extensões necessárias para suportar subprogramas.

Abaixo apresenta-se uma versão simplificada da gramática na notação BNF (*Backus-Naur Form*), agrupada pelos principais blocos estruturais da linguagem.

3.1 Especificação BNF

1. Estrutura Global e Subprogramas

```

<programa>          ::= <bloco_prog> <lista_subprog>
<bloco_prog>         ::= PROGRAM ID <declaracoes> <comandos> END
<lista_subprog>      ::= <subprog> <lista_subprog> | vazia
<subprog>            ::= <def_subroutine> | <def_function>
<def_subroutine> ::= SUBROUTINE ID ( <parametros> ) <declaracoes> <comandos>

```

```

<def_function> ::= <tipo> FUNCTION ID ( <parametros> )
                <declaracoes> <comandos> RETURN END

```

2. Declarações e Tipos

```

<declaracoes> ::= <declaracao> <declaracoes> | vazia
<declaracao>  ::= <tipo> <lista_vars>
<tipo>        ::= INTEGER | REAL | LOGICAL
<lista_vars>  ::= ID | ID , <lista_vars> | ID ( NUM )

```

3. Comandos e Controlo de Fluxo

```

<comandos>    ::= <comando> <comandos> | vazia
<comando>     ::= [LABEL] <instrucao>
<instrucao>   ::= <atribuicao>
                | <if_bloco>
                | <ciclo_do>
                | GOTO LABEL
                | CONTINUE
                | READ * , <lista_vars>
                | PRINT * , <lista_expr>
                | CALL ID ( <lista_expr> )

<atribuicao>   ::= ID = <expressao> | ID ( <expressao> ) = <expressao>
<if_bloco>    ::= IF ( <expressao> ) THEN <comandos> <opt_else> ENDIF
<opt_else>    ::= ELSE <comandos> | vazia
<ciclo_do>    ::= DO LABEL ID = <expressao> , <expressao> [ , <expressao> ]

```

4. Expressões (Aritméticas, Relacionais e Lógicas)

```

<expressao> ::= <expressao> + <expressao>
                | <expressao> - <expressao>
                | <expressao> * <expressao>
                | <expressao> / <expressao>
                | <expressao> ** <expressao>
                | ( <expressao> )
                | <expressao> .EQ. <expressao> | <expressao> .NE. <expressao>
                | <expressao> .LT. <expressao> | <expressao> .GT. <expressao>
                | <expressao> .AND. <expressao> | <expressao> .OR. <expressao>
                | .NOT. <expressao>
                | - <expressao>
                | ID | NUM | STR | .TRUE. | .FALSE.
                | ID ( <lista_expr> )

```

3.2 Resolução de Ambiguidades e Precedência

A tradução direta de uma gramática de expressões e controlo de fluxo para regras de produção introduz, por norma, ambiguidades clássicas que o algoritmo LALR(1) não consegue resolver nativamente sem gerar conflitos de *Shift/Reduce*. No desenvolvimento do *parser*, estas ambiguidades foram tratadas utilizando as diretivas de precedência do *PLY*:

- **Precedência de Operadores:** Foi definida uma tabela de precedência (variável *precedence* no *Yacc*). Os operadores lógicos (*.OR.*, *.AND.*) têm a menor precedência, seguidos dos operadores relacionais, e por fim os aritméticos. A exponenciação (****) foi definida com associatividade à direita e precedência superior à da multiplicação e divisão, enquanto o "menos" (*uminus*) teve precedência máxima.
- **O Problema do *Dangling Else*:** O conflito associado aos blocos *IF-THEN-ELSE* aninhados foi resolvido dizendo ao gerador para optar pelo *Shift* em vez do *Reduce* quando encontra um *token ELSE*. Desta forma, garante-se a semântica padrão da maioria das linguagens de programação: um *ELSE* pertence sempre ao *IF* mais recente e ainda aberto.

4 Testes Implementados

4.1 Casos de Teste

Os ficheiros de teste foram organizados no diretório *tests/* e dividem-se nas seguintes categorias:

- **Testes Padrão:** Foram implementados os cinco programas de exemplo fornecidos no guião do projeto para testar as funcionalidades básicas:
 - *hello.f77*: Validação de instruções de *output* básicas (*PRINT*).
 - *fatorial.f77*: Teste de ciclos *DO* com contadores e operações aritméticas multiplicativas.
 - *primo.f77*: Verificação de tipagem lógica (*LOGICAL*), condições compostas (*.AND.*) e saltos incondicionais (*GOTO*).
 - *somaarr.f77*: Validação da declaração, iteração e acesso a posições de memória de *arrays* unidimensionais.
 - *conversor.f77*: Teste à definição de funções (*FUNCTION*), escopos de variáveis locais e passagem de parâmetros.
- **Testes Adicionais e Casos Limite:** Para garantir a robustez de operações complexas que exigiram soluções criativas na VM, o grupo desenvolveu testes específicos:
 - *pow_test.f77*: Valida o cálculo correto da exponenciação (****) através de multiplicações sucessivas *inline*.
 - *maxmin_test.f77*: Verifica a avaliação correta das funções intrínsecas *MAX* e *MIN* através de saltos condicionais na VM.
 - *fibonacci.f77*: Um caso de teste extra que combina ciclos e reatribuição contínua de variáveis locais.

4.2 Automação da Bateria de Testes

Foi criado o *script run_tests.py*. Esta ferramenta de automação executa o seguinte fluxo para cada ficheiro *.f77* presente na pasta de testes:

1. Invoca o compilador (`main.py`) para processar o código fonte.
2. Interceta eventuais erros de compilação (sintáticos ou semânticos) e reporta-os.
3. Guarda o código VM gerado nos ficheiros `.vm` correspondentes em `tests/expected/`.

Esta abordagem permite validar rapidamente se o compilador processa todos os ficheiros de teste sem erros de compilação.

5 Instruções de Utilização

5.1 Requisitos de Instalação

O compilador depende da biblioteca *PLY* (*Python Lex-Yacc*). Caso não esteja instalada, pode ser obtida através do gestor de pacotes *pip*:

```
pip install ply
```

5.2 Compilação de Ficheiros

O ponto de entrada do compilador é o ficheiro `main.py` localizado na raiz do projeto. Para compilar um ficheiro Fortran (`.f77`), deve utilizar o seguinte comando:

```
python main.py tests/fibonacci.f77
```

Este processo executa sequencialmente o pré-processamento, as análises léxica, sintática e semântica e, finalmente, a geração de código. Se o código fonte for válido, será gerado um ficheiro com a extensão `.vm` no mesmo diretório (ex: `tests/fibonacci.vm`).

5.3 Execução na Máquina Virtual

Para executar o código intermédio gerado, utiliza-se o interpretador da Máquina Virtual fornecido:

```
python vm/vm.py tests/fibonacci.vm
```

5.4 Execução dos Testes

Foi incluído um *script* de testes para validar o compilador face a diversos cenários de teste. Para correr todos os testes presentes na pasta `tests/`, execute:

```
python tests/run_tests.py
```

Este comando compila todos os ficheiros de exemplo e verifica a ausência de erros de compilação. Para executar também os programas na VM e observar o *output*, utilize a opção `--run`:

```
python tests/run_tests.py --run
```

6 Dificuldades Encontradas

O desenvolvimento do compilador confrontou o grupo com desafios técnicos específicos, principalmente associados à diferença entre as abstrações do *Fortran 77* e as restrições da Máquina Virtual (VM) de destino.

A coexistência de formatos (*fixed-form* e *free-form*) e a insensibilidade a espaços exigiram a criação de um pré-processador customizado, o que tornou o projeto mais trabalhoso. Para além disso, a ausência de instruções nativas na VM para exponenciação (******) e para as funções intrínsecas **MAX** e **MIN** foi superada através da geração de blocos de código *inline*, tirando partido de saltos condicionais e da alocação de endereços de memória temporários. Por fim, garantir a coerência estrutural da linguagem, como a verificação obrigatória de que cada *label* de um ciclo **DO** possui um comando **CONTINUE** correspondente, exigiu a implementação de uma fase de reestruturação pós-parse dedicada (`src/structure.py`), com lógica de varrimento recursivo que suporta ciclos **DO** aninhados.

7 Conclusão

O projeto resultou no desenvolvimento bem-sucedido de um compilador funcional para o standard *Fortran 77*, que cumpre os requisitos estruturais, metodológicos e de correção exigidos. A modularidade aplicada ao *pipeline* em Python, pré-processador, análise léxica, análise sintática, reestruturação da AST, análise semântica e geração de código, permitiu isolar eficazmente cada fase de tradução, resultando num sistema de fácil manutenção.

Para além do núcleo essencial da linguagem, o grupo alcançou uma valorização importante ao estender o suporte do compilador para a definição e invocação de subprogramas (FUNCTION e SUBROUTINE), gerindo dinamicamente os *frames* de ativação na pilha da Máquina Virtual através de instruções de alocação local.